

Кодирование файла — инженерный разбор

Тот же материал, что в академической монографии `EncoderGuide.Academic.ru.md`, но без формализмов ради формализмов: короче записи, больше «почему сделано именно так» и «как это ведёт себя на реальных данных». Нумерация разделов в обеих версиях совпадает, так что ссылки взаимозаменяемы. Вводный разбор «с нуля» — `EncoderGuide.Tutorial.ru.md`; контекст формата — в `AlgorithmGuide.ru.md` (разделы 2–4). Обратный процесс — `DecoderGuide.Engineer.ru.md`.

1. Что мы вообще строим

`FileEncoder` превращает файл в список 75-байтных пакетов: `T` секторов данных ($N \text{ data} + M \text{ ECC}$) плюс `H` копий заголовка. Два параметра конструктора: `eccPercent` (по умолчанию 10) и `headerPercent` (по умолчанию 3). Всё остальное — производные. API: `Encode` (байты или `Stream`), `EncodeToText` (Base64 по строке на пакет), `EncodeWithStats` (+сводка `EncodeStats`). Прогресс глобальный 0–100, отмена — `CancellationToken`.

2. Модель потерь и избыточность

Поток переживает два вида повреждений: потерю кусков (лечится ECC и повторами заголовков) и мусор/рассинхронизацию (лечится скользящим окном декодера). Отсюда два процента:

- **ЕСС** — сколько data-томов можно потерять безвозвратно: $M = \max(1, \lceil N \cdot p / 100 \rceil)$. Потеря любых $k \leq M_{\text{avail}}$ data-томов восстанавливаема; потерянные ECC-тома бесплатны.
- **Заголовки** — живучесть опознавания файла: $H = \max(3, \lceil T \cdot h / 100 \rceil)$. Три копии всегда: начало, конец, середина.

На маленьких файлах проценты работают как «минимум один/три» — избыточность кратная, но это цена гарантии.

3. Константы формата

Константа	Значение	Смысл
<code>PacketSize</code>	75	неделимый пакет
<code>PayloadSize</code>	64	D2
<code>HeaderContentSize</code>	51	H1–H4
<code>HeaderHashSize</code>	24	H5, 192 бита
<code>SectorHashSize</code>	9	D3, 72 бита
<code>MaxFileSizeField</code>	16 777 215	потолок файла (поле 3 байта)
<code>MaxDataVolumes</code>	65 535	потолок N+M ($GF(2^{16})$)

Превышения — `InvalidOperationException`, а не молчаливая усечённая выдача.

4. Подготовка данных

Порядок: проверки → `FileNameCodec.Pack(Path.GetFileName(name))` → `Sha256Compact.HashData(content)` → размерности:

```
N = max(1, [FileSize/64]); M = max(1, [N·ecc%/100]) или 0; T = N+M
```

Нарезка: N payload по 64 байта, последний добивается нулями. Вайп промежуточных буферов после сборки пакетов: содержимое файла не должно задерживаться в памяти дольше необходимого.

Имя в 14 байт: режим по первой точке, расширение (включая `.tar.gz`) сохраняется целиком, квантуется база, при усечении добавляется `~`. Бюджет базы < 2 байт — отказ. Отказ на этапе упаковки имени — это правильно: лучше исключение сейчас, чем заголовок с пустым/обрезанным до неузнаваемости именем.

5. RS-кодирование

`RsCodecAdapter.Encode(dataPayloads, M)`: 64-байтный том = 32 символа $GF(2^{16})$ (UInt16 LE); для каждой из 32 позиций собирается столбец из N data-символов, `Rsrail16.Process` достраивает M ЕСС-символов — линейных комбинаций над полем. Позиции независимы, прогресс тикает по символам. Проверка `K+M ≤ 65535` до старта. Сама математика поля — эталонный `Rsrail16`, не трогается.

Свойства на выходе: любой набор из `M` доступных ЕСС покрывает `M` потерянных data; восстановление — точная линейная алгебра, без переборов и вероятностей.

6. Заголовочный пакет

`HeaderContent` сериализуется в 51 байт (H1 имя, H2 размер LE, H3 SHA-256, H4 M LE), затем `H5` `= Trunc24(SHA-256(51 байт))` — и пакет готов: 51+24. Двойная роль H5: автономная самопроверка заголовка (декодеру не нужно ничего знать заранее) и сид для D3 всех секторов файла. Усечение до 192 бит достаточно: ложное срабатывание $\approx 2^{-192}$.

7. Секторы данных

Каждый payload → сектор `D1(2 LE) || D2(64) || D3(9)`, где `D3 = Trunc9(SHA-256(H5 || D1 || D2))`. Вход хеша — 90 байт. Ключевой инвариант: **без H5 сектор не проверить**, поэтому чужие пакеты к файлу не приклеиваются, а свои опознаются только после заголовка (проблема «сектор раньше заголовка» решается на стороне декодера адресной перепривязкой — `DecoderGuide.Engi`
`neer.ru.md`, раздел 10).

8. Расстановка заголовков

`ArrangePackets(headerPacket, sectors, H)`: первый и последний пакеты — заголовки; промежуточные `H-2` копии вставляются после каждого `[T/(H-1)]`-го сектора. Равномерность означает: вырезание любого одного интервала потока оставляет ≥ 2 копий с вероятностью, близкой к единице на реальных T. Порядок секторов — натуральный: перемешивание не нужно, окна декодера сами переживают рассинхронизацию.

9. Выходные форматы и PacketIO

- **Base64** (`.DataShield.txt`): строка = пакет = ровно 100 символов, `=` не бывает (75 кратно 3). Декоративные оформления `>[имя] [размер] [SHA-256:hex] / <[...]` — для человека, декодер их отбрасывает фильтром.
- **Binary** (`.DataShield.bin`): сырые пакеты подряд.

`PacketIO.WriteFile` пишет файл или поток; `GetDefaultOutputPath` = вход + расширение формата; `DetectFormat` на чтении смотрит на расширение (нераспознанное → Base64).

10. Прогресс и отмена

Фазы глобальной шкалы: подготовка 0–10, ECC 10–75 (масштабируется `ScaledProgress`), пакетизация 75–100, финал `Done`. Троттлинг по целым процентам (`ProgressThrottle`). Отмена проверяется между фазами и внутри символьного цикла RS; исключение наружу, полупоток не выдаётся.

11. Статистика и вайпы

`EncodeWithStats` возвращает `EncodeStats(FileSize, Sha256, DataCount, EccCount, TotalPackets, HeaderCopies)` — всё, что нужно интерфейсу для отчёта «что получится». После `Encode` обнуляются: data payload, ECC payload, байты заголовка. Пакеты-результат живут как есть — в них содержимое неизбежно присутствует.

12. Поточковый вход

`ReadStreamContent`: позиционируемый поток читается в массив по остатку (`Length - Position`, проверка потолка), непозиционируемый — через `MemoryStream` копию. Поток не закрывается, работает с любым `Stream`, включая `MemoryStream`. Полное чтение — не каприз: SHA-256 и RS-кодирование требуют всего содержимого.

13. Сквозной пример

Файл 1000 байт, есс 10%, заголовки 3%: `N=16`, `M=2`, `T=18`, копий заголовка `max(3, [0.54])=3`, интервал 9 → поток `H D0...D8 H D9...D17 H`, 20 пакетов: 1500 байт binary или 20×100 символов Base64. Последний data-том: 40 байт данных + 24 нуля. Для больших N расширение сходится к ≈132% (binary) и ≈176% (Base64).

14. Параметры и пределы

Параметр	Диапазон	Эффект
<code>eccPercent</code>	0...100 (далее упрётся в GF-потолок)	М томов; 0 = без восстановления
<code>headerPercent</code>	0...100	Н копий; всегда ≥ 3
Размер файла	≤ 16 777 215	поле H2
<code>N+M</code>	≤ 65 535	поле GF(2 ¹⁶)

Памятка: `eccPercent=10` теряет безвозвратно только при потере > 10% data-томов и соответствующих ECC; `headerPercent=3` даёт копию заголовка примерно на каждые 33 сектора.

15. Краевые случаи

- Пустой файл → `N=1` (один нулевой том), собирается корректно.
- `eccPercent=0` → ECC-фаза пропускается полностью (нулевой overhead).
- 1-байт файл с ECC → `N=1, M=1, T=2`, избыточность 6400% — гарантия работает.
- Длинное расширение (база < 2 байт) → отказ на упаковке имени.
- Файл за потолками → явные исключения с числами в сообщении.

16. Что гарантировано

1. Либо полный корректный поток, либо исключение — промежуточных состояний нет.
2. **H3** — точный SHA-256; арбитраж сборки безупречен.
3. Каждый сектор привязан сидом **H5**; подмена не проходит проверку D3.
4. Потеря $\leq M$ data-томов восстановима при выживших ЕСС (точная алгебра, не эвристика).
5. Минимум 3 копии заголовка, равномерно: начало/середина/конец.
6. Вайп промежуточных буферов: лишних копий содержимого в памяти не остаётся.